HDOC-35

Aim:

To write a menu-driven C program that reads a variable-length square integer matrix and performs the following operations using separate functions:

1. Check whether all main diagonal elements are distinct.
2. Find the sum of the main diagonal elements.
3. Display the matrix elements using alternating diagonal traversal.
4. Exit.

Task 1: Algorithm:

Step 1: Start the program.

Step 2: Read the order n of the square matrix.

Step 3: Declare the variable-length array:

int a[n][n];

Step 4: Read n × n integer elements into the matrix.

Step 5: Display the menu repeatedly until the user selects Exit.

For Choice 1 — Distinct Main Diagonal

- Access diagonal elements using a[i][i].
- Compare each diagonal element with all subsequent diagonal elements.
- If any two are equal, report that they are not distinct.
- Otherwise, report that all are distinct.

For Choice 2 — Diagonal Sum

- Initialize sum = 0.
- Traverse from i = 0 to n-1.

- Add a[i][i] to sum.
- Return and display the sum.

For Choice 3 — Diagonal Traversal

- There are 2n - 1 diagonals.
- Number them from 0 to 2n - 2.
- For even-numbered diagonals, traverse **upward-right**.
- For odd-numbered diagonals, traverse **downward-left**.
- Print each element encountered.

For Choice 4: Exit the program.

Step 6: Stop.

Task 2: Test Case Table:

Test Case	Matrix	Choice	Expected Output

1	1 2 3
	`4 5 6`
	`7 8 9`
		1	Main diagonal: 1 5 9 → **All distinct**

2	1 2 3
	`4 1 6`
	`7 8 9`
		1	Main diagonal: 1 1 9 → **Not distinct**

3	1 2 3
	`4 5 6`
	`7 8 9`
		2	Sum = **15**

4	1 2 3
	`4 5 6`
	`7 8 9`
		3	1 2 4 7 5 3 6 8 9

5	1 2 `3 4`	3	1 2 3 4

Task 3: C Program:

```
#include <stdio.h>
// Function prototypes
void checkDistinct(int n, int a[n][n]);
int diagonalSum(int n, int a[n][n]);
void diagonalTraversal(int n, int a[n][n]);
int main()
{
  int n, choice;
  printf("Enter the order of the square matrix: ");
  scanf("%d", &n);
  // Validate matrix size
  if (n <= 0)
  {
    printf("Matrix size must be positive.\n");
    return 0;
  }
  // Variable Length Array
  int a[n][n];
  printf("Enter the matrix elements:\n");
  for (int i = 0; i < n; i++)
  {
    for (int j = 0; j < n; j++)
    {
      scanf("%d", &a[i][j]);
    }
  }
  do
  {
    printf("\n----- MENU -----\n");
    printf("1. Check if main diagonal elements are distinct\n");
    printf("2. Find sum of main diagonal elements\n");
    printf("3. Diagonal traversal\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
      case 1:
        checkDistinct(n, a);
        break;
      case 2:
        printf("Sum of main diagonal elements = %d\n",
            diagonalSum(n, a));
        break;
      case 3:
```

```
            diagonalTraversal(n, a);
            break;
        case 4:
            printf("Exiting program...\n");
            break;
        default:
            printf("Invalid choice! Please enter 1 to 4.\n");
    }
  } while (choice != 4);
  return 0;
}
// Function to check whether main diagonal elements are distinct
void checkDistinct(int n, int a[n][n])
{
  printf("Main diagonal elements: ");
  for (int i = 0; i < n; i++)
  {
    printf("%d ", a[i][i]);
  }
  printf("\n");
  for (int i = 0; i < n - 1; i++)
  {
    for (int j = i + 1; j < n; j++)
    {
      if (a[i][i] == a[j][j])
      {
        printf("All the main diagonal elements are NOT distinct.\n");
        return;
      }
    }
  }
  printf("All the main diagonal elements are distinct.\n");
}
// Function to calculate sum of main diagonal elements
int diagonalSum(int n, int a[n][n])
{
  int sum = 0;
  for (int i = 0; i < n; i++)
  {
    sum = sum + a[i][i];
  }
  return sum;
}
// Function for alternating diagonal traversal
void diagonalTraversal(int n, int a[n][n])
{
  printf("Diagonal traversal: ");
```

```
    // There are 2*n - 1 diagonals
    for (int d = 0; d <= 2 * n - 2; d++)
    {
        // Even diagonal: upward-right
        if (d % 2 == 0)
        {
            int i;
            if (d < n)
                i = d;
            else
                i = n - 1;
            int j = d - i;
            while (i >= 0 && j < n)
            {
                printf("%d ", a[i][j]);
                i--;
                j++;
            }
        }
        // Odd diagonal: downward-left
        else
        {
            int j;
            if (d < n)
                j = d;
            else
                j = n - 1;
            int i = d - j;
            while (j >= 0 && i < n)
            {
                printf("%d ", a[i][j]);
                i++;
                j--;
            }
        }
    }
    printf("\n");
}
```

Task 4: Output:

```
┌──(kali㉿kali)-[~/hdoc35]
└─$ ./a.out
Enter the order of the square matrix: 3
Enter the matrix elements:
1
2
3
4
5
6
7
8
9

——— MENU ———
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct.

——— MENU ———
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements = 15

——— MENU ———
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9

——— MENU ———
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting program ...
```

```
┌──(kali㉿kali)-[~/hdoc35]
└─$ ./a.out
Enter the order of the square matrix: 3
Enter the matrix elements:
1
2
3
4
1
6
7
8
9

──── MENU ────
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 1 9
All the main diagonal elements are NOT distinct.

──── MENU ────
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting program...
┌──(kali㉿kali)-[~/hdoc35]
└─$ ./a.out
Enter the order of the square matrix: 2
Enter the matrix elements:
1
2
3
4

──── MENU ────
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 3 4

──── MENU ────
1. Check if main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting program...
```